

```
git > aston-lessons > module_2 > src > Student.java > Student > getBooks()
1  package src;
2
3  import java.util.List;
4
5  public class Student {
6      private String name;
7      private List<Book> books;
8
9      public Student(String name, List<Book> books) {
10         this.name = name;
11         this.books = books;
12     }
13
14     public String getName() {
15         return name;
16     }
17
18     public List<Book> getBooks() {
19         return books;
20     }
21
22     @Override
23     public String toString() {
24         return name + ": книги=" + books;
25     }
26 }
27
```

```

1 package src;
2
3 public class Book {
4     private String title;
5     private int pages;
6     private int year;
7
8     public Book(String title, int pages, int year) {
9         this.title = title;
10        this.pages = pages;
11        this.year = year;
12    }
13
14    /* Getters
15     */
16    public String getTitle() { return title; }
17    public int getPages() { return pages; }
18    public int getYear() { return year; }
19
20    @Override
21    public boolean equals(Object o) {
22        if (this == o) return true;
23        if (o == null || getClass() != o.getClass()) return false;
24        Book book = (Book) o;
25        return pages == book.pages && year == book.year && title.equals(book.title);
26    }
27
28    @Override
29    public int hashCode() {
30        return title.hashCode() + pages + year;
31    }
32
33    @Override
34    public String toString() {
35        return String.format("%s' (%d стр., %d г.)", title, pages, year);
36    }
37 }
38

```

```

package src;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class Main {
    public static void main(String[] args) {

        InputStream is = Main.class.getResourceAsStream("db.txt");

        if (is == null) {

```

```
System.err.println("Error: файл db.txt не найден в classpath");
return;
}

List<Student> students = new ArrayList<>();

try (BufferedReader reader = new BufferedReader(new InputStreamReader(is))) {
    List<String> lines = reader.lines().toList();

    String name = null;
    List<Book> books = new ArrayList<>();

    for (String line : lines) {
        line = line.trim();

        if (line.isEmpty()) {
            /* Если пустая строка разделитель
            */
            if (name != null) {
                /* Если имя студента уже определено,
                *   добавляем его в список
                */
                students.add(new Student(name, new ArrayList<>(books)));
                /* Очищаем список книг
                */
                books.clear();
                /* Очищаем имя студента
                */
                name = null;
            }
        } else if (name == null) {
            /* Если имя студента еще не определено,
            *   запоминаем студента
            */
            name = line;
        } else {
            /* Если строка не пустая и имя студента определено,
            *   добавляем книгу в список
            */

            /* Разбиваем строку с книгами на части
            */
            String[] parts = line.split(":");
            if (parts.length == 3) {
                /* Если строка с книгой соответствует формату
                */
                String title = parts[0].trim();
                int pages = Integer.parseInt(parts[1].trim());
                int year = Integer.parseInt(parts[2].trim());
                books.add(new Book(title, pages, year));
            }
        }
    }
}
```

```

    }
}

/* После цикла добавляем последнего студента в список
*/
if (name != null) {
    students.add(new Student(name, books));
}
} catch (IOException e) {
    System.err.println("Ошибка чтения файла: " + e.getMessage());
}

/* Вывод для проверки
*/
// System.out.println("=== Студенты из файла ===");
// students.forEach(System.out::println);
// System.out.println();

/* Основное задание через один стрим
*/

System.out.println("=== Результат обработки одним стримом ===");

students.stream()
    .peek(System.out::println)
    .flatMap(student -> student.getBooks().stream())
    .distinct()
    .filter(book -> book.getYear() > 2000)
    .sorted(Comparator.comparing(Book::getPages))
    .limit(3)
    .map(Book::getYear)
    .findFirst()
    .ifPresentOrElse(
        year -> System.out.println("Год выпуска найденной книги: " + year),
        () -> System.out.println("Такая книга отсутствует")
    );
}
}

```